

UNIVERSITY INSTITUTE OF COMPUTING



PROJECT REPORT

Program Name: BCA

Subject Name: JAVA PROGRAMMING LAB

Submitted by:

Name: Mehakdeep kaur

UID: 24BCA10232

Section: BCA – 7 “B”

Submitted to:

Name: Mrs. Renu Jangra



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

QS WORLD
UNIVERSITY
RANKINGS
RANKED 1ST
AMONGST PVT. UNIVERSITIES IN INDIA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their valuable guidance, support, and encouragement throughout the development of this project titled *"Snake Game using Java Swing with Real-Time Score Tracking."* Their continuous motivation and expert advice helped me in successfully completing this project.

I am also thankful to my college for providing the necessary resources and a conducive environment to carry out this work effectively. The knowledge and skills I have gained during my coursework played an important role in the successful implementation of this project.

I would like to extend my heartfelt thanks to my friends and classmates for their support, suggestions, and cooperation during the development process.

Lastly, I express my deep gratitude to my family for their constant encouragement and moral support, which helped me stay focused and complete this project on time.

Introduction

The Snake Game is a classic arcade game that has been popular for decades due to its simple gameplay and engaging mechanics. In this game, the player controls a snake that moves continuously within a bounded area. The primary objective of the game is to collect food items that appear randomly on the screen. Each time the snake consumes food, its length increases, and the player's score improves.

This project implements the Snake Game using the Java programming language along with the Swing library for graphical user interface (GUI) development. The game demonstrates how basic programming concepts can be applied to create an interactive and real-time application. It also provides an understanding of how user inputs are handled using keyboard events and how animations are created using timers.

The game environment consists of a grid-based layout where the snake moves in four directions: up, down, left, and right. The player must carefully control the snake to avoid collisions with the walls or with its own body. If such a collision occurs, the game ends, and the final score is displayed.

Objective

The main objective of this project is to design and develop an interactive Snake Game using Java Swing that provides a user-friendly graphical interface and real-time gameplay experience. The project aims to demonstrate how programming concepts can be applied to create a functional and engaging game.

One of the primary goals is to understand and implement graphical user interface components using the Swing library. This includes creating windows, panels, and handling graphics for drawing the snake and food. Another important objective is to implement event handling using keyboard inputs so that the player can control the movement of the snake effectively.

The project also focuses on implementing real-time score tracking, where the score increases as the snake consumes food. This helps in understanding how dynamic data is updated and displayed on the screen during execution. Additionally, the project aims to introduce the concept of a game loop using timers, which ensures continuous movement and smooth gameplay.

Finally, the project encourages the development of problem-solving skills by handling challenges such as collision detection, random food generation, and maintaining the snake's body structure. Overall, this project serves as a practical application of Java programming concepts.

Tools and Technologies Used

The Snake Game project is developed using a combination of programming tools and technologies that support graphical user interface design and real-time interaction. The primary programming language used in this project is Java, which is widely known for its platform independence, object-oriented nature, and extensive library support.

For designing the graphical user interface, the Swing library is used. Swing is a part of Java Foundation Classes (JFC) and provides a set of components such as frames, panels, buttons, and graphics utilities that help in building interactive desktop applications. It allows developers to create visually appealing interfaces with ease.

The Abstract Window Toolkit (AWT) is also used in this project for handling events such as keyboard inputs. AWT provides classes and methods that enable communication between the user and the application, making it possible to control the snake using arrow keys.

The project also utilizes built-in Java classes such as Timer for controlling the speed of the game and Random for generating food positions. These technologies work together to create a smooth and interactive gaming experience. Overall, the combination of Java, Swing, and AWT makes this project efficient and easy to implement.



**CHANDIGARH
UNIVERSITY**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

QS WORLD
UNIVERSITY
RANKINGS
RANKED 1ST
AMONGST PVT. UNIVERSITIES IN INDIA

System Design

The system design of the Snake Game focuses on creating a structured and efficient approach to implement the game logic and user interface. The game is designed using a grid-based system where the entire game area is divided into small equal-sized units. The snake moves from one unit to another in a continuous manner, giving the appearance of smooth motion.

The design consists of several key components, including the game panel, snake body representation, food generation, and user input handling. The game panel is responsible for rendering all graphical elements, including the snake, food, and score. The snake is represented using arrays that store the x and y coordinates of each segment of its body.

Food is generated randomly within the boundaries of the game area using a random number generator. When the snake's head reaches the same position as the food, the food is consumed, and a new food item is generated at a different location.

User input is handled using a key listener that detects arrow key presses and changes the direction of the snake accordingly. A timer is used to create a game loop, which repeatedly updates the position of the snake, checks for collisions, and redraws the screen.

Implementation

The Snake Game is implemented using Java with Swing and AWT for graphical interface and event handling. A JFrame is used to create the main window, and a JPanel is used to draw the snake, food, and score.

The snake is represented using arrays that store the positions of its body parts. A Timer is used to control the game loop, which continuously updates the movement of the snake, checks for food consumption, and detects collisions.

User input is handled using keyboard arrow keys to change the direction of the snake. Food is generated at random positions using the Random class. When the snake eats the food, its length increases and the score is updated.

Collision detection is used to check if the snake hits the wall or itself. If a collision occurs, the game stops and a “Game Over” message is displayed.

This implementation ensures smooth gameplay and demonstrates the use of GUI, event handling, and game logic in Java.

Code

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.Random;

public class SnakeGame extends JPanel implements ActionListener {

    static final int WIDTH = 600;
    static final int HEIGHT = 600;
    static final int UNIT_SIZE = 25;
    static final int GAME_UNITS = (WIDTH * HEIGHT) / UNIT_SIZE;
    static final int DELAY = 100;

    final int x[] = new int[GAME_UNITS];
    final int y[] = new int[GAME_UNITS];

    int bodyParts = 6;
    int applesEaten;
    int appleX;
    int appleY;

    char direction = 'R';
    boolean running = false;
    Timer timer;
    Random random;

    SnakeGame() {
        random = new Random();
        this.setPreferredSize(new Dimension(WIDTH, HEIGHT));
        this.setBackground(Color.black);
        this.setFocusable(true);
```



```
this.addKeyListener(new MyKeyAdapter());
startGame();
}

public void startGame() {
    newApple();
    running = true;
    timer = new Timer(DELAY, this);
    timer.start();
}

public void paintComponent(Graphics g) {
    super.paintComponent(g);
    draw(g);
}

public void draw(Graphics g) {
    if (running) {
        g.setColor(Color.red);
        g.fillOval(appleX, appleY, UNIT_SIZE, UNIT_SIZE);

        for (int i = 0; i < bodyParts; i++) {
            if (i == 0) {
                g.setColor(Color.green);
                g.fillRect(x[i], y[i], UNIT_SIZE, UNIT_SIZE);
            } else {
                g.setColor(new Color(45,180,0));
                g.fillRect(x[i], y[i], UNIT_SIZE, UNIT_SIZE);
            }
        }

        g.setColor(Color.white);
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.drawString("Score: " + applesEaten, 10, 20);
    } else {
```

```
        gameOver(g);
    }
}

public void newApple() {
    appleX = random.nextInt((int)(WIDTH / UNIT_SIZE)) * UNIT_SIZE;
    appleY = random.nextInt((int)(HEIGHT / UNIT_SIZE)) * UNIT_SIZE;
}

public void move() {
    for (int i = bodyParts; i > 0; i--) {
        x[i] = x[i - 1];
        y[i] = y[i - 1];
    }

    switch(direction) {
        case 'U': y[0] -= UNIT_SIZE; break;
        case 'D': y[0] += UNIT_SIZE; break;
        case 'L': x[0] -= UNIT_SIZE; break;
        case 'R': x[0] += UNIT_SIZE; break;
    }
}

public void checkApple() {
    if (x[0] == appleX && y[0] == appleY) {
        bodyParts++;
        applesEaten++;
        newApple();
    }
}

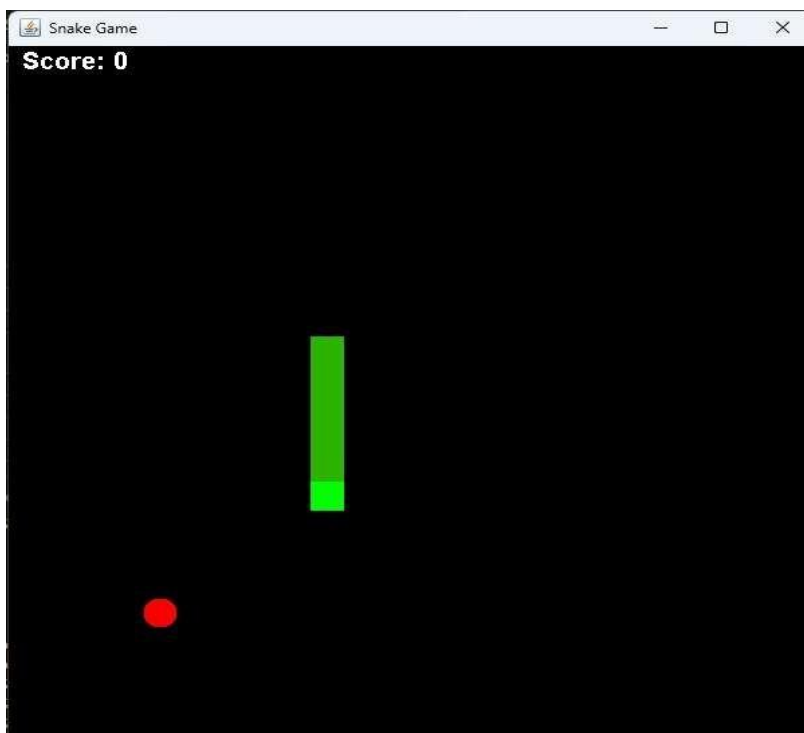
public void checkCollisions() {
    for (int i = bodyParts; i > 0; i--) {
        if (x[0] == x[i] && y[0] == y[i]) {
            running = false;
        }
    }
}
```

```
    }  
}  
  
if (x[0] < 0 || x[0] >= WIDTH || y[0] < 0 || y[0] >= HEIGHT) {  
    running = false;  
}  
  
if (!running) timer.stop();  
}  
  
public void gameOver(Graphics g) {  
    g.setColor(Color.red);  
    g.setFont(new Font("Arial", Font.BOLD, 40));  
    g.drawString("Game Over", WIDTH/4, HEIGHT/2);  
  
    g.setFont(new Font("Arial", Font.BOLD, 20));  
    g.drawString("Score: " + applesEaten, WIDTH/3, HEIGHT/2 + 40);  
}  
  
@Override  
public void actionPerformed(ActionEvent e) {  
    if (running) {  
        move();  
        checkApple();  
        checkCollisions();  
    }  
    repaint();  
}  
  
public class MyKeyAdapter extends KeyAdapter {  
    public void keyPressed(KeyEvent e) {  
        switch(e.getKeyCode()) {  
            case KeyEvent.VK_LEFT: if (direction != 'R') direction = 'L'; break;  
            case KeyEvent.VK_RIGHT: if (direction != 'L') direction = 'R'; break;  
            case KeyEvent.VK_UP: if (direction != 'D') direction = 'U'; break;
```

```
        case KeyEvent.VK_DOWN: if (direction != 'U') direction = 'D'; break;
    }
}
}
```

```
public static void main(String[] args) {
    JFrame frame = new JFrame("Snake Game");
    SnakeGame game = new SnakeGame();
    frame.add(game);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.pack();
    frame.setVisible(true);
    frame.setLocationRelativeTo(null);
}
```

OUTPUT





CONCLUSION

The Snake Game project successfully demonstrates the practical implementation of Java programming concepts in a real-world application. By developing this game, we gain a deeper understanding of graphical user interface design, event handling, and real-time processing. The use of Java Swing makes it possible to create an interactive and visually appealing game environment.

Throughout the project, various challenges such as controlling the movement of the snake, detecting collisions, and generating random food positions were addressed effectively. These challenges helped in improving logical thinking and problem-solving skills. The implementation of score tracking added an extra layer of interactivity, making the game more engaging for users.

The project also highlights the importance of structured programming and modular design. By dividing the program into different methods and components, the code becomes easier to understand, debug, and maintain. This approach is essential for developing larger and more complex applications in the future.

In conclusion, the Snake Game project is an excellent example of how basic programming knowledge can be used to create a functional and enjoyable application. It serves as a valuable learning experience for students and beginners in Java programming.